

REKAYASA PERANGKAT LUNAK

Modul Pembelajaran: Konsep Dasar Rekayasa Perangkat Lunak

Penjelasan komprehensif mengenai definisi, karakteristik, evolusi, hingga pilar utama dalam pengembangan perangkat lunak secara profesional.

Contents

1	Pengantar Perangkat Lunak (Software)	2
1.1	Definisi Perangkat Lunak	2
1.2	Karakteristik Perangkat Lunak	2
1.3	Karakteristik Perangkat Lunak yang Baik	2
2	Apa itu Rekayasa Perangkat Lunak (RPL)?	3
2.1	Definisi RPL (<i>Software Engineering</i>)	3
2.2	Falsafah: Why, Where, When, dan How?	3
3	Cakupan dan Aktivitas Utama RPL	4
3.1	Tiga Ranah Utama (Pilar RPL)	4
3.2	Proses Aktivitas Dasar Profesional	4
4	Evolusi dan Jenis Aplikasi Perangkat Lunak	6
4.1	Evolusi Perangkat Lunak	6
4.2	Berbagai Kategori Aplikasi Perangkat Lunak	6

1 Pengantar Perangkat Lunak (Software)

Sebelum kita membahas konsep "Rekayasa" atau *Engineering*, kita harus menyamakan persepsi dulu tentang apa itu sebenarnya "Perangkat Lunak" (*Software*).

1.1 Definisi Perangkat Lunak

Banyak orang awam mengira perangkat lunak itu sekadar baris-baris kode program (kodingan). Padahal, dalam kacamata profesional, perangkat lunak jauh lebih luas dari itu. Menurut standar IEEE (1993), perangkat lunak adalah program komputer, prosedur, aturan, dan dokumentasi yang berkaitan, serta data, yang saling berhubungan dengan operasi suatu sistem komputer.

Secara sederhana, rumusnya adalah:

$$\text{Perangkat Lunak} = \text{Program} + \text{Data} + \text{Dokumentasi}$$

1.2 Karakteristik Perangkat Lunak

Tidak seperti perangkat keras (*hardware*) yang bisa dirakit di pabrik dan lama-kelamaan bisa rusak berkarat, perangkat lunak punya sifat alami yang sangat unik:

1. **Dibangun dengan rekayasa (*Engineered*):** Perangkat lunak itu dikembangkan atau direkayasa melalui pemikiran logis, bukan diproduksi secara massal atau dicetak pakai mesin pabrik layaknya *hardware*.
2. **Tidak pernah usang (*Doesn't wear out*):** Kode program tidak bisa berkarat, aus, atau terdegradasi secara fisik karena cuaca atau gesekan. Kalau ada *error*, itu biasanya karena cacat logika dari awal, bukan karena komponennya "aus".
3. **Terus berevolusi:** Perangkat lunak akan terus menerus diperbaiki, di-*update*, atau dimodifikasi seiring dengan bertambahnya kebutuhan pengguna atau perubahan lingkungan sistem.

1.3 Karakteristik Perangkat Lunak yang Baik

Sebuah perangkat lunak baru bisa dikatakan "sukses" dan "berkualitas" jika memenuhi kriteria-kriteria berikut:

- **Usability:** Mudah dipelajari dan digunakan oleh *user*.
- **Good Performance:** Responsif, cepat, dan tidak *lag*.
- **Be Reliable:** Andal, konsisten, dan tidak gampang *crash* atau mati mendadak.
- **Maintainability:** Mudah dirawat, kodenya rapi, sehingga gampang kalau mau di-*update* atau nambah fitur baru.
- **Efficient:** Tidak boros dalam menggunakan sumber daya (*memory*, CPU, atau baterai).
- **Eye Catching User Interface:** Tampilan antarmukanya menarik dan memanjakan mata.
- **Long Life Time:** Punya masa pakai yang panjang dan relevan untuk waktu yang lama.

2 Apa itu Rekayasa Perangkat Lunak (RPL)?

2.1 Definisi RPL (*Software Engineering*)

Rekayasa Perangkat Lunak (RPL) atau *Software Engineering* (SE) adalah satu bidang profesi yang secara khusus mendalami cara-cara pengembangan perangkat lunak secara profesional, mulai dari pembuatan, pemeliharaan, hingga manajemen kualitasnya.

Ada dua definisi formal yang sangat populer di kalangan akademisi dan praktisi:

- **Menurut IEEE Computer Society:**

RPL adalah penerapan pendekatan yang sistematis, disiplin, dan terkuantifikasi (dapat diukur) atas pengembangan, penggunaan, dan pemeliharaan perangkat lunak, serta studi atas pendekatan-pendekatan tersebut. Singkatnya: *menerapkan prinsip rekayasa (engineering) ke dalam dunia perangkat lunak.*

- **Menurut Ian Sommerville:**

RPL adalah disiplin rekayasa yang berkaitan dengan semua aspek produksi perangkat lunak, mulai dari tahap paling awal (spesifikasi sistem) sampai dengan pemeliharaan sistem ketika sistem tersebut sudah mulai digunakan oleh klien.

Perhatian Penting!

Rekayasa Perangkat Lunak (*Software Engineering*) itu **TIDAK SAMA** dengan sekadar Pengembangan Perangkat Lunak (*Software Development*). *Development* mungkin sekadar ngoding dan bikin aplikasi jalan. Tapi *Engineering* melibatkan standar kedisiplinan tingkat tinggi, prosedur yang ketat, manajemen risiko, dan jaminan mutu!

2.2 Falsafah: Why, Where, When, dan How?

Untuk memahami urgensi dari disiplin ilmu ini, mari kita bedah melalui empat pertanyaan krusial:

- **Why? (Kenapa harus ada RPL?)**

Pengembangan perangkat lunak dengan **skala besar** tidak mungkin bisa dikerjakan oleh satu orang (semisal *lone-wolf programmer*) saja. Proyek besar butuh kolaborasi dan kerja tim yang baik agar bisa selesai tepat waktu dan sesuai *budget*.

- **Where? (Di mana RPL diterapkan?)**

RPL utamanya mutlak diperlukan pada proyek pengembangan perangkat lunak berskala menengah (*medium*) hingga besar (*large scale*).

- **When? (Kapan RPL digunakan?)**

Ketika pembuatan *software* tidak bisa lagi ditangani pakai cara-cara "tradisional" (asal ngoding). Terutama ketika proyek tersebut mensyaratkan target kualitas, tenggat waktu (*deadline*), dan efisiensi yang super ketat.

- **How? (Bagaimana cara menerapkannya?)**

Melalui penerapan teknik-teknik rekayasa terstruktur yang mencakup empat fase hidup: Spesifikasi (menentukan apa yang mau dibuat), Pengembangan (ngoding), Validasi (uji coba), dan Evolusi (perawatan/update).

3 Cakupan dan Aktivitas Utama RPL

3.1 Tiga Ranah Utama (Pilar RPL)

Disiplin ilmu dasar Rekayasa Perangkat Lunak berdiri kokoh di atas perpotongan tiga ranah utama. Jika salah satu ranah ini diabaikan, proyek *software* kemungkinan besar akan hancur lebur.

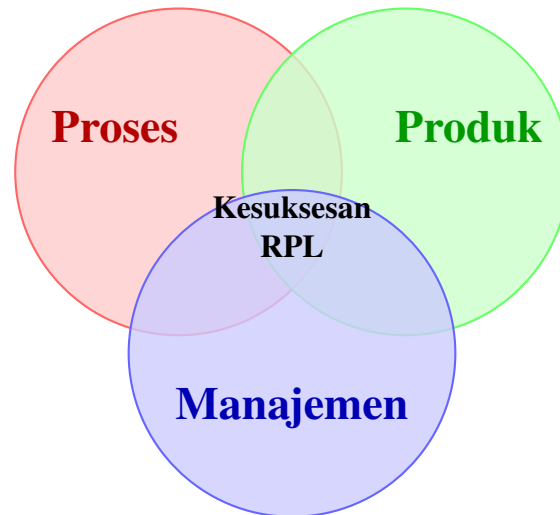


Figure 1: Tiga Pilar Utama Pembentuk Disiplin Rekayasa Perangkat Lunak.

1. **Proses:** Mengacu pada langkah-langkah, metode, dan *framework* (kerangka kerja) teknis yang digunakan untuk membangun sistem.
2. **Produk:** Wujud nyata dari *software* yang dihasilkan, baik itu kode, desain arsitektur, hingga dokumentasinya. Harus terukur kualitasnya.
3. **Manajemen:** Urusan mengelola sumber daya manusia (tim), mengatur jadwal, mengatur biaya (*budgeting*), dan memastikan risiko terkendali.

3.2 Proses Aktivitas Dasar Profesional

Dalam pengembangan perangkat lunak secara profesional, ada empat aktivitas mendasar yang selalu dilakukan secara sekuensial atau iteratif:

1. **Analisis (Spesifikasi):** Mencari tahu dan merumuskan apa yang sebenarnya dibutuhkan oleh pelanggan (*requirements*).
2. **Perancangan (Desain):** Menerjemahkan kebutuhan tadi menjadi cetak biru (*blueprint*) arsitektur sistem, basis data, dan antarmuka.
3. **Implementasi:** Menulis baris kode program (*coding*) berdasarkan desain yang sudah dibuat.
4. **Pengujian (Testing):** Memastikan kode yang dibuat bebas dari **bug** dan benar-benar berjalan sesuai keinginan pelanggan.

Selain aktivitas dasar di atas, pembelajaran RPL secara utuh akan mencakup berbagai topik spesifik seperti: Metode Pengembangan, Manajemen Proyek, Pembuatan Dokumen Spesifikasi, Pemodelan Diagram (UML: *Use Case*, *Activity*, *Sequence*, *Class*), Perancangan Antarmuka (UI/UX), hingga Penjaminan Kualitas (QA).

4 Evolusi dan Jenis Aplikasi Perangkat Lunak

Seiring berjalannya waktu, karakteristik perangkat lunak yang dikembangkan oleh manusia mengalami perubahan dan lompatan yang sangat drastis.

4.1 Evolusi Perangkat Lunak

Perkembangan historis *software* dapat dibagi menjadi empat era besar:

Timeline Era	Karakteristik Utama
Era 1 (1950 - 1960)	<i>Beberapa Tahun Lalu.</i> Perangkat lunak murni berorientasi <i>batch</i> (diproses berbarengan di akhir). Distribusinya sangat terbatas dan hanya dibuat kustom (<i>custom software</i>) untuk perusahaan tertentu saja.
Era 2 (1960 - 1970)	Sudah mulai mendukung fitur <i>Multiuser</i> dan bekerja secara seketika (<i>Real-time</i>). Di era ini pula konsep Basis Data (<i>Database</i>) mulai muncul, serta software mulai dijual sebagai produk kemasan (<i>Product software</i>).
Era 3 (1970 - 1990)	Masuk ke fase sistem terdistribusi. Munculnya kecerdasan yang ditanamkan (<i>Embedded "intelligence"</i>) pada perangkat keras berbiaya rendah. Perangkat lunak mulai memberi dampak masif bagi konsumen rumah tangga.
Era 4 (1990 - 2000an)	Komputer meja (<i>Desktop</i>) menjadi sangat canggih. Lahirnya teknologi berorientasi objek (<i>Object-oriented technologies</i>), sistem pakar (<i>Expert systems</i>), Jaringan Saraf Tiruan (AI), komputasi paralel, hingga sistem berbasis Jaringan/Internet.

Table 1: Garis Waktu Evolusi Perangkat Lunak.

4.2 Berbagai Kategori Aplikasi Perangkat Lunak

Hari ini, perangkat lunak menopang hampir seluruh aspek kehidupan manusia. Berdasarkan fungsinya, *software* bisa dikategorikan menjadi 9 jenis utama:

1. **System Software:** Program yang melayani program lain (cth: Sistem Operasi seperti Windows, Linux).
2. **Real-time Software:** Memonitor dan merespon peristiwa di dunia nyata seketika itu juga (cth: sistem navigasi pesawat).
3. **Business Software:** Mengelola data bisnis dan operasional perusahaan (cth: sistem penggajian, *point of sales*).
4. **Engineering and Scientific Software:** Perhitungan kompleks untuk ilmuwan (cth: simulasi astronomi, CAD).
5. **Embedded Software:** *Software* yang ditanam ke dalam perangkat keras spesifik (cth: *software* di dalam mesin cuci atau *microwave*).

6. **Artificial Intelligence (AI) Software:** Memecahkan masalah rumit tanpa algoritma perhitungan pasti (cth: *machine learning*, pengenalan suara).
7. **Internet Software:** Aplikasi yang diakses melalui peramban web.
8. **Software Tools and CASE environment:** *Software* yang dipakai untuk membantu programer membuat *software* lain (cth: IDE, compiler).
9. **Produk Developer:** Aplikasi generik (bisa dipakai siapa saja, cth: MS Word) atau aplikasi kustom (pesanan khusus satu perusahaan).